

PerDiem

An AI agent that spends your per-diem — and cannot cross the line.

Mingyu Lee · MICEMore

PerDiem is a policy layer that holds a traveler's per-diem budget and permitted-merchant list for one business trip, stops any agent payment that falls outside it before it reaches the chain, and produces receipts an auditor can verify from records alone.

- Kiln API · qwen3-32b on Furiosa NPUs

- Policy engine in code · 12 checks

- Ethereum Sepolia · anchored mandate + receipts

- github.com/keilvher1/perdiem

The moment after delegation

Payment rails record **who paid whom** — not who authorized it, or under what conditions.

Good behavior is an accident

An agent that stays within budget does so only because someone happened to build it that way.

Nothing to reconstruct

When something goes wrong, nobody can tell whether the agent stayed inside the line it was given.

Refusals are silent

There is no record that the agent was stopped, or why — so stopping cannot be trusted either.

PerDiem makes delegated spending answerable: rules are set once, enforced in code before every payment, refusals are recorded, and a third party can verify every payment from the records alone.

Delegate the per-diem without losing control

FM

Finance manager at a small MICE company sending a colleague to a two-day conference in Seoul. The traveler asks an AI agent for lunch, taxis and supplies; finance sets the rules once and must answer for every payment afterwards.

WHY

I have settled government startup-grant expenses by hand: rules in prose, boxes ticked by hand, refusals weeks later as free text. **PerDiem does the opposite:** rules enforced in code before money moves, each refusal recorded with its code, observed value and limit, checks recomputed from the records.

01 · GRANT

Set the mandate

Budget \$150, \$40 per payment, meals · transport · supplies, five permitted merchants, blocked words, trip window. Hashed and anchored on Sepolia.

/principal

02 · FOLLOW

Watch the spend

Live spend gauge and ledger. A broadcast payment counts as committed while it is still pending on-chain.

/principal · /traveler

03 · STOP

Pause at any time

Pause, Resume, Revoke. Every request after Pause is stopped with `MANDATE_NOT_ACTIVE` and recorded.

PATCH /api/mandates/[id]

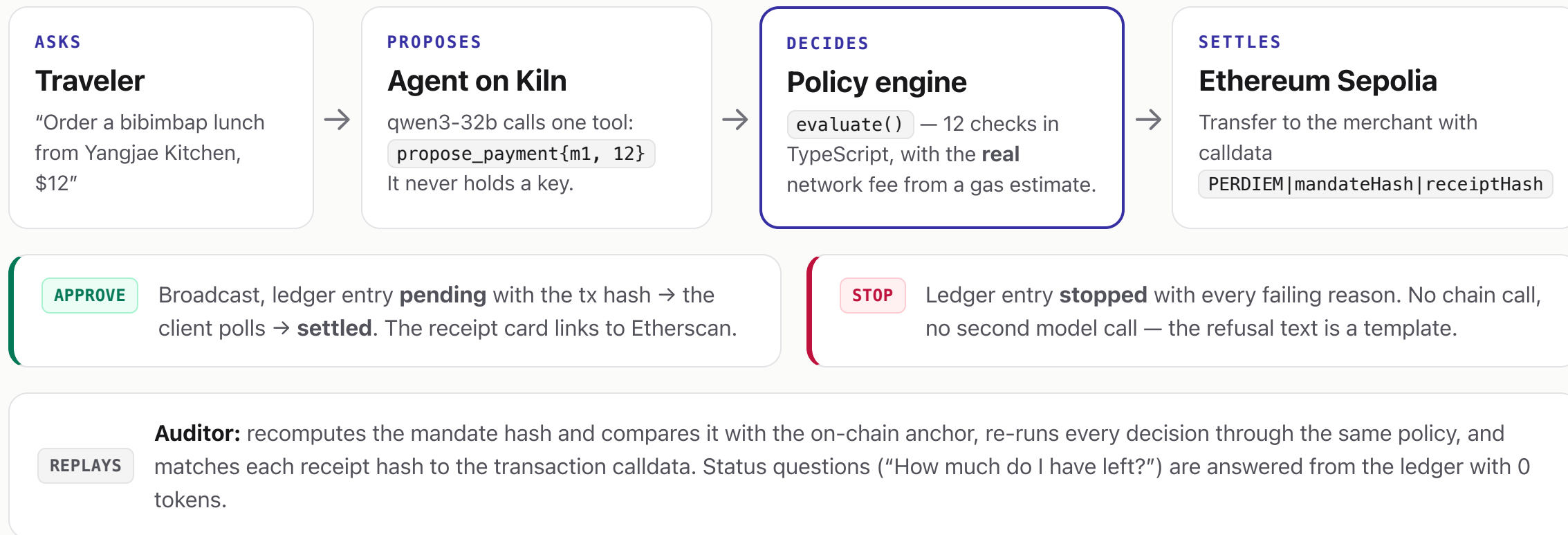
04 · VERIFY

Audit from records

Download the mandate and ledger from the **Evidence** button; re-check every payment with a public RPC — no access to our app.

Evidence · verify.ts · /audit

The model proposes. Code decides. The chain settles.



Kiln applies `tool_choice: auto only` — no tool call ⇒ no proposal ⇒ nothing is spent

Only `lib/chain.ts` touches the wallet, only after APPROVE

Twelve checks, in code, before anything reaches the chain

`lib/policy.ts · evaluate()` is a pure function called in `lib/agent.ts` before any call into `lib/chain.ts`. Every failing check is reported, not just the first.

MANDATE_NOT_ACTIVE

Principal paused or revoked the mandate — the kill switch.

BEFORE_START

The trip window has not opened yet.

EXPIRED

The deadline has already passed.

UNKNOWN_MERCHANT

Not in the catalog snapshot hashed with the mandate.

MERCHANT_NOT_ALLOWED

In the catalog, but not on the permitted list.

CATEGORY_NOT_ALLOWED

The merchant's category is not permitted.

BLOCKED_KEYWORD

Memo or the traveler's words mention a blocked item.

INVALID_AMOUNT

Zero, negative, or not a number.

OVER_PER_TX_CAP

Above the single-payment cap.

FEE_UNAVAILABLE

Fee could not be estimated → refuse rather than guess.

OVER_BUDGET_WITH_FEES

Amount + real network fee exceeds what is left. Never rounded.

DUPLICATE

Same merchant and amount within five minutes.

The model never holds keys

Unit-tested · `npm test` (9 suites)

Mandate hash covers terms + catalog, not status

Stopping is recorded, not silent

Real runs on Kiln and Sepolia — approvals and refusals

Approved. Paying Yangjae Kitchen \$12.00 for "Order a bibimbap lunch" (est. network fee \$0.115177).

Approved — inside the mandate

Settled

Yangjae Kitchen

meal · m1 · "Order a bibimbap lunch"

You said: "Order a bibimbap lunch from Yangjae Kitchen, \$12"

\$12.00

Amount

\$12.00

Network fee estimate

\$0.115177

Actual fee after mining

\$0.11963

Counted against budget

\$12.115177

tx

0x1d7a...0767

receipt

0x5af8...c4e0

mandate

0xd743...6a36

Recorded in ledger as `Led_1790601009634_5d9m` · 22:10:09 · settled 22:10:26

Stopped — nothing was sent

Stopped

Wine & Co

alcohol · m5 · "Buy a bottle of wine as a gift for the client"

You said: "Buy a bottle of wine as a gift for the client, \$30"

\$30.00

requested · \$0.00 sent

3 of 12 boundary checks failed

[MERCHANT_NOT_ALLOWED]

Wine & Co is not on the permitted list.

Observed m5

Limit m1, m2, m3, m4, m7

[CATEGORY_NOT_ALLOWED]

Category "alcohol" is not permitted.

Observed alcohol

Limit meal, transport, supplies

[BLOCKED_KEYWORD]

Request mentions a blocked item ("wine").

Observed wine

Recorded in ledger as `Led_1790601030982_1quz` · 22:10:30 · no transaction broadcast

Stopped. I did not pay Yangjae Kitchen \$10.00.

Stopped — nothing was sent

Stopped

Yangjae Kitchen

meal · m1 · "Dinner from Yangjae Kitchen"

You said: "Dinner from Yangjae Kitchen, \$10"

\$10.00

requested · \$0.00 sent

1 of 12 boundary checks failed

[OVER_BUDGET_WITH_FEES]

Amount \$10.00 + network fee \$0.11556 = \$10.11556 exceeds remaining budget \$10.00.

Observed \$10.11556

Limit \$10.00

Recorded in ledger as `Led_1790601036057_kquw` · 22:10:36 · no transaction broadcast

receipt

0x968c...0b8b

mandate

0x63bf...4b89

#0 Lunch \$12 — approved, settled on Sepolia

#2 Wine gift \$30 — stopped with three reasons

#3 \$10 dinner, \$10 budget — \$10 + fee > \$10.00

#	TRAVELER SAYS	MANDATE	OUTCOME	REASONS / TRANSACTION
0	Order a bibimbap lunch from Yangjae Kitchen, \$12	A	APPROVE	0x23ab6a82...ffa4 · settled
1	Taxi to Incheon airport, about \$85	A	STOP	OVER_PER_TX_CAP
2	Buy a bottle of wine as a gift for the client, \$30	A	STOP	MERCHANT_NOT_ALLOWED, CATEGORY_NOT_ALLOWED, BLOCKED_KEYWORD
3	Dinner from Yangjae Kitchen, \$10	B	STOP	OVER_BUDGET_WITH_FEES
4	Coffee at Starbucks, \$5 (while paused)	A	STOP	MANDATE_NOT_ACTIVE
5	Coffee at Starbucks, \$5	C	STOP	EXPIRED
6	Coffee at Starbucks, \$5 (after resume)	A	APPROVE	0x3f9ce201...dfff · settled
7	How much do I have left?	A	0 TOKENS	answered from the ledger, no model call

Recorded run 2026-09-28 20:33 KST (man*_ev1) — 8/8 outcomes as expected · evidence/scenario-20260928-2033.json · screenshots above: chat take man*_ev3, 22:10 KST

/audit/<mandate> — the same checks, in the browser

- ```
$ npm run verify -- evidence/mandate-<id>.json \
 evidence/ledger-<id>.json
mandate man_A_ev1 - recomputed hash 0xab2cdb88...215c
✅ anchor 0x00bfdf66...edaf memo matches recomputed hash
✅ anchor sent by the mandate's agent wallet 0x8340daD3...29eF
✅ led_1790595191309_pnhq: entry carries the mandate hash
✅ led_1790595191309_pnhq: stored APPROVE == recomputed APPROVE
... 20 more checks
ALL RECORDS VERIFIED
```

PerDiem · Challenge B

# Only one model call per purchase — and no hidden reasoning

154 → 47

completion tokens per proposal, thinking off (-69%)

3.0 → 1.2 s

latency per proposal · cost -48%

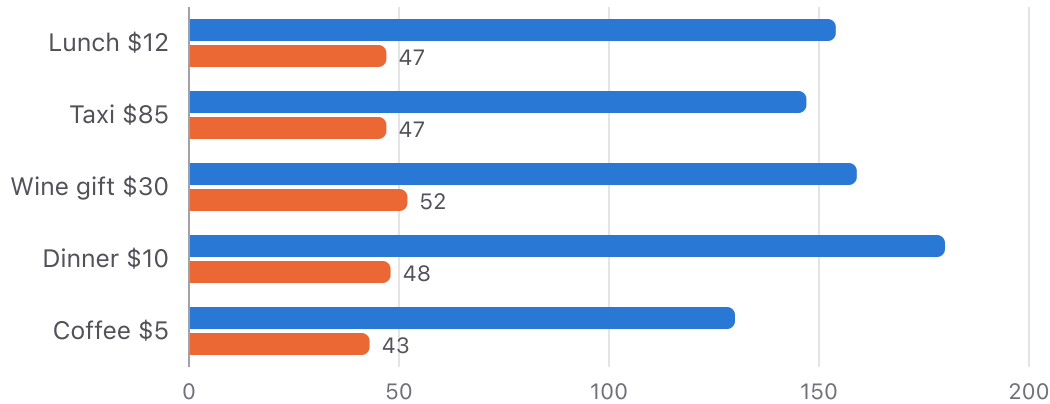
5/5 / 5/5

correct tool calls, thinking on / off

### Completion tokens per purchase request

Kiln qwen3-32b, production prompt and tool, measured 2026-09-28 18:12 KST · thinking off = /no\_think

Thinking on Thinking off (production)



| FLOW            | CALLS | PROMPT | COMPLETION | COST (USD) |
|-----------------|-------|--------|------------|------------|
| propose         | 21    | 9,828  | 969        | \$0.000685 |
| status_fastpath | 3     | 0      | 0          | \$0.000000 |
| stop_template   | 15    | 0      | 0          | \$0.000000 |
| compare         | 10    | 4,676  | 1,007      | \$0.000471 |

/api/usage at 2026-09-28 22:11 KST; compare = the thinking on/off run (docs/reasoning-comparison.json), outside the server. Green rows never call the model.

- Status questions answered from the ledger — 0 tokens
- Refusals explained from a template — 0 tokens
- Compact id | name | category catalog, one tool call per turn
- Thinking switched off for the propose step, same accuracy

**Energy:** ≈ 1.29 Wh for 10,797 tokens = tokens × 0.429 J/token ÷ 3600.  
**Assumption:** one RNGD card (180 W TDP) busy for a proposal's measured 1.233 s over its 517 tokens. Multi-card serving or host power would raise it; batching and network time would lower it. An estimate, not a measurement: Kiln does not report per-request energy.

# From a test-ETH demo to a finance tool

## Stablecoin settlement

The same policy and audit with an ERC-20 transfer (test USDC) and a real FX feed instead of the labeled demo rate.

## Smart-account mandates

The principal signs the mandate; the agent wallet becomes a smart account with a session key scoped to it. Multi-traveler trips.

## Receipts finance can import

Export the verified ledger in a format expense tools accept, with the on-chain proof attached to every line.

## Run it yourself

```
git clone https://github.com/keilvher1/perdiem && cd perdiem
cp .env.example .env.local # Kiln key, dev wallet, Supabase
Supabase SQL editor: run docs/schema.sql
npm ci && npm run seed -- --window now && npm run dev
npm run scenario # the 8 scripted runs
```

## Honest scope

Testnet only, no real money, no auth. Pause and resume show as log lines and MANDATE\_NOT\_ACTIVE entries, not as hashed records. Pre-kickoff modules (contract, policy engine, agent loop, Kiln and chain wrappers, verifier, tests) are disclosed in the README; the app, routes, scripts and this deck were built during the event with AI coding agents.

# Thank you.

Rendered 2026-09-28 22:29 KST from docs/deck/deck.html